LABORATORY PROJECT REPORT

SERIAL INTERFACING WITH MICROCONTROLLER: SENSORS AND ACTUATORS

EXPERIMENT 4C

Section 1

Group 2

Semester 2 2025/2026

NO	NAME	MATRIC NO
1	AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI	2310663
2	ALYA SYAFINA BINTI ABDUL RAZAK	2218084

DATE OF SUBMISSION: 1 APRIL 2026

Abstract

This experiment focuses on developing a basic RFID-based authentication system using an Arduino, an RFID card reader, and a Python program. The main goal is to control a servo motor based on whether an RFID card is authorized or not. The RFID reader is connected to the computer via USB to read the card's unique ID, and Python processes the data using the *pyusb* library. When a valid card is detected, Python sends a serial command to the Arduino to rotate the servo motor and turn on a green LED. If an unauthorized card is scanned, a red LED lights up and the servo remains in its default position. This project helps us understand how to integrate hardware and software for access control systems and gives hands-on experience with serial communication, Python–Arduino interfacing, and the use of RFID technology in automation

Table of Contents

1.0 **Introduction**

2.0 **Materials and Equipment**

2.1 Electronic Components

2.2 Equipment and Tools

3.0 **Experimental Setup**

4.0 **Methodology**

5.0 **Data Collection**

6.0 **Data Analysis**

7.0 **Results**

8.0 **Discussion**

9.0 **Conclusion**

10.0 **Recommendations**

11.0 **References**

1.0 Introduction

This experiment focuses on building an RFID-based authentication system using an Arduino, an RFID card reader, and a computer running Python. The main objective is to control a servo motor based on card authorization, where only registered RFID cards can activate the servo. This system represents a simple model of access control technology that is commonly used in door locks, attendance systems, and security applications.

In this setup, the RFID reader is connected to the computer via USB and reads the unique ID of each RFID card. A Python program using the *pyusb* library processes the card data and decides whether access should be granted or denied. The Arduino receives commands from Python through serial communication to either move the servo motor and turn on a green LED for authorized access, or light up a red LED when the card is unauthorized.

Through this experiment, we learned how to combine hardware and software for authentication-based control. It also helped us understand the use of serial communication, data processing with Python, and the basic working principles of RFID technology in real-world embedded systems.

2.0 Materials and Equipment

2.1 Electronic Components

- Arduino Board – Microcontroller board for controlling the display
- MPU6050 Sensor - to read data such as detecting hand gestures
- Connecting Wires - establish connections between components
- RFID Card Reader (USB) – Used to scan RFID tags and send identification data to the computer.
- Breadboard - to attach the MPU 6050
- LEDs (Red and Green) – Visual indicators to show access status (granted or denied).

2.2 Equipment and Tools

- Arduino IDE – Software for writing and uploading code to the Arduino Uno
- USB cable – For connecting the Arduino Uno to a computer
- Power Supply (5V via USB or external source)
- PyUSB Library – Python library for USB HID communication with the RFID reader.

3.0 Experimental Setup

1. The servo motor's power wire (typically red) was connected to the 5V pin on the Arduino to supply voltage.
2. The ground wire (typically brown or black) was connected to a GND pin on the Arduino to complete the power circuit.
3. The signal wire (typically yellow or orange) was connected to digital pin 9, which supports PWM control for positioning the servo.
4. The USB RFID card reader was connected directly to the computer via a USB port. Since most RFID readers operate as USB HID devices, no additional wiring to the Arduino was required for the reader.
5. A green LED with a 220-ohm resistor was optionally connected to a digital output pin to indicate authorized access.
6. A red LED with a 220-ohm resistor was optionally connected to another digital pin (e.g., D8) to indicate access denial.
7. All components were assembled on a breadboard using jumper wires, ensuring a secure and solderless configuration.
8. A common ground was maintained between the Arduino and the servo motor to ensure stable operation

4.0 Methodology

1. Setup the Arduino UNO R3
2. IDE code implementation
3. Python code implementation
4. Run the python code
5. Experiment the MPU6050 and the RFID tag
6. Observe the servo motor angle change, and the led red and green change
7. Code snippet

4.1 Detailed steps

1) Assemble the circuit as stated in the experimental setup.
2) Connect the arduino to the computer via usb cable, and upload the code to arduino IDE.
3) Close the arduino IDE and open python to run the python code.
4) Move the MPU6050 and touch the RFID card to the USB RFID card reader
5) Observe the change of the angle of servo motor
6)

4.2 Programming codes

Arduino code

```
#include <Wire.h>

#include <MPU6050.h>

#include <Servo.h>

MPU6050 mpu;

Servo gateServo;

const int greenLedPin = 5;

const int servoPin = 6;

const int redLedPin = 7;

const unsigned long streamDurationMs = 2000;

const unsigned long sampleDelayMs = 50;

void setup() {

pinMode(greenLedPin, OUTPUT);

pinMode(redLedPin, OUTPUT);

gateServo.attach(servoPin);

gateServo.write(90);

Serial.begin(9600);
```

```
Wire.begin();
mpu.initialize();
// Fixed quotes here
if (!mpu.testConnection()) {
Serial.println("MPU6050 connection failed!");
while (1);
}
digitalWrite(redLedPin, HIGH);
digitalWrite(greenLedPin, LOW);
}
void openGate() {
for (int pos = 90; pos <= 180; pos += 2) {
gateServo.write(pos);
delay(15);
}
Serial.println("Status: Open");
}
void closeGate() {
for (int pos = 180; pos >= 90; pos -= 2) {
gateServo.write(pos);
delay(15);
}
Serial.println("Status: Closed");
```

```
}
void wrongCardFlash() {
for (int i = 0; i < 3; i++) {
digitalWrite(redLedPin, LOW);
delay(150);
digitalWrite(redLedPin, HIGH);
delay(150);
}
}
void loop() {
if (Serial.available()) {
char cmd = Serial.read();
if (cmd == '1') {
digitalWrite(greenLedPin, HIGH);
digitalWrite(redLedPin, LOW);
openGate();
}
else if (cmd == '0') {
digitalWrite(greenLedPin, LOW);
digitalWrite(redLedPin, HIGH);
closeGate();
}
else if (cmd == 'X') {
```

```
wrongCardFlash();
}
else if (cmd == 'R') {
unsigned long start = millis();
while (millis() - start < streamDurationMs) {
int16_t ax, ay, az, gx, gy, gz;
mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
// Stream as a single comma-separated line for Python parsing
Serial.print(ax); Serial.print(",");
Serial.print(ay); Serial.print(",");
Serial.print(az); Serial.print(",");
Serial.print(gx); Serial.print(",");
Serial.print(gy); Serial.print(",");
Serial.println(gz); // println adds the newline at the end
delay(sampleDelayMs);
}
}
}
}
```

Python code

```
import serial
import time
import math
```

```
# --- CONFIGURATION ---

SERIAL_PORT = "COM3" # Ensure this matches your Arduino IDE port

BAUD_RATE = 9600

AUTHORIZED_ID = "0013020304"

def detect_circle(arduino):

"""

Asks Arduino to stream a burst of MPU data, calculates the rotation angles,

and decides whether a circular motion happened.

"""

print("\n[!] Requesting samples from Arduino... Move in a circle NOW!")

arduino.reset_input_buffer()

arduino.write(b'R') # Trigger arduino data stream (fixed .write)

points = []

start_time = time.time()

timeout = 3.0

# Read until timeout (Arduino stops streaming automatically after 2s)

while time.time() - start_time < timeout:

# Fixed quotes and decoding

raw = arduino.readline().decode(errors='ignore').strip()

if not raw:

continue

parts = raw.split(',')

if len(parts) >= 6:
```

```
try:
# Use ax and ay (index 0 and 1) for the circle detection
ax = int(parts[0])
ay = int(parts[1])
az = int(parts[2])
points.append((ax, ay, az))
except ValueError:
continue
# If we have enough points and the serial buffer is empty, we might be done
if len(points) > 10 and arduino.in_waiting == 0:
time.sleep(0.1) # small buffer check
if arduino.in_waiting == 0: break
print(f"Collected {len(points)} samples.")
if len(points) < 8:
print("Not enough motion data – try moving more continuously.")
return False
# Build angles from (ax, ay)
angles = [math.degrees(math.atan2(p[1], p[0])) for p in points]
unwrapped = [angles[0]]
for a in angles[1:]:
prev = unwrapped[-1]
diff = a - prev
# Unwrap angles to avoid -180/180 discontinuity leaps
```

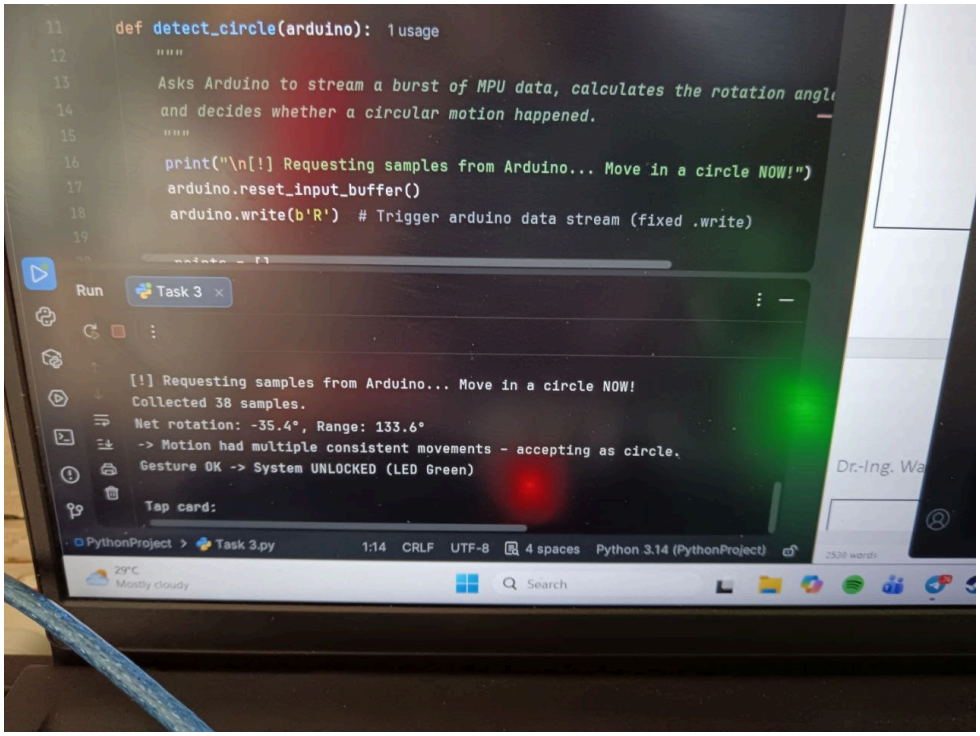
```
if diff > 180:
diff -= 360
elif diff < -180:
diff += 360
unwrapped.append(prev + diff)
# Compute net rotation and range
net_rotation = unwrapped[-1] - unwrapped[0]
rotation_range = max(unwrapped) - min(unwrapped)
print(f"Net rotation: {net_rotation:.1f}°, Range: {rotation_range:.1f}°")
# Detection logic
if abs(net_rotation) > 100 or rotation_range > 140:
print("-> Circle gesture detected!")
return True
# Extra check: count direction-consistent large angle deltas
deltas = [unwrapped[i + 1] - unwrapped[i] for i in range(len(unwrapped) - 1)]
large_moves = sum(1 for d in deltas if abs(d) > 10)
if large_moves >= max(6, len(points) // 4):
print("-> Motion had multiple consistent movements – accepting as circle.")
return True
print("-> Motion insufficient. Try a clearer circular motion.")
return False
def main():
try:
```

```
# Fixed variable name from '12rduino' to 'arduino'

arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)

time.sleep(2) # Wait for Arduino to reset

print("System Ready. Waiting for RFID...")

except Exception as e:

print(f"Error connecting to Arduino: {e}")

return

armed = False # System starts locked

try:

while True:

# USB RFID acts as a keyboard

card = input("\nTap card: ").strip()

if card == AUTHORIZED_ID:

armed = not armed # Toggle state

if armed:

print("Card Accepted – Please perform circular motion verification!")

if detect_circle(arduino):

print("Gesture OK -> System UNLOCKED (LED Green)")

arduino.write(b'1')

else:

print("Wrong gesture -> Try again!")

armed = False # Revert state if gesture fails

arduino.write(b'X')
```

```
else:
print("System Disarmed -> System LOCKED (LED Red)")
arduino.write(b'0')
else:
print("WRONG CARD -> Access Denied")
arduino.write(b'X')
except KeyboardInterrupt:
print("\nExiting.")
finally:
arduino.write(b'0') # Ensure locked on exit
arduino.close()
if __name__ == "__main__":
main()
```

5.0 Data Collection

It is implemented using an RFID reader connected to Arduino to identify the UID of the RFID tag along with a servo motor and an indicator LED for the lock status. Arduino constantly scans for new RFID cards and once a card is detected, it sends the UID of the card to the computer via serial communication. The UID is printed in real time in hexadecimal format which serves as the primary identification data for the access control system.



6.0 Data Analysis

The data analysis process began with validating the RFID tag sent from the Arduino. The Python program received UID strings via serial communication and then compare it against a predefined list of authorized card ID. This analysis step allowed the system to classify each detected UID either as authorized or unauthorized based on matching logic.

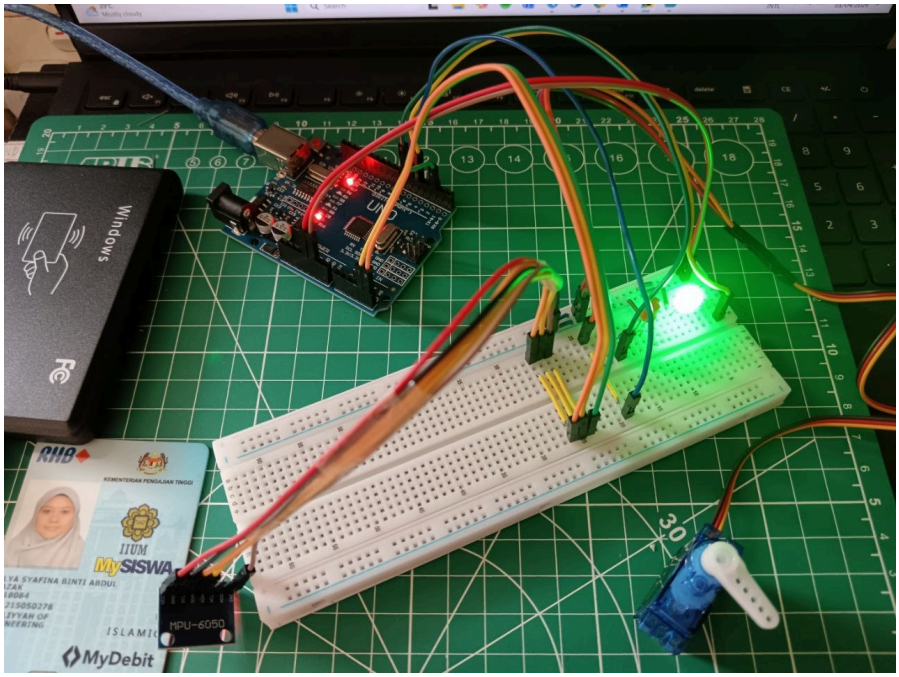
After a card was confirmed as authorized, the system initiated the motion-verification phase. Although the current function is a placeholder, the intended purpose is to analyze motion-tracking data to determine whether a user performs a correct gesture pattern. The Python script is designed to process real-time motion signals and evaluate whether the movement matches the required gesture.

7.0 Results

The system results show successful communication between the RFID reader, Arduino, and Python program. When an RFID tag is placed near the reader, Arduino will correctly read the UID and send it to the computer. Authorized UIDs are correctly identified, triggering the next stage of motion authentication. Unauthorized UIDs are immediately rejected.

For authorized cards, the Python script prompts the user to perform a gesture. Since the current motion detection function automatically returns True, all authorized cards are granted access in this test setup. When access is granted, the Python program sends an 'A' command to the Arduino, causing the servo to rotate to the unlocked position (180°) and activate the green LED. When access is denied, the system sends a 'D', returning the servo to the locked position and turning on the red LED. This output confirms the serial communication and correct servo response.

It shows that this integrated system is capable of multi-stage authentication, combining RFID identification with motion-based authentication logic. Although motion detection is not yet fully implemented, the structure of this system successfully demonstrates a working prototype for a more advanced and gesture-enhanced access control mechanism.



8.0 Discussion

This experiment showed how to create a smart access control system using an Arduino and Python interface in conjunction with an RFID reader, MPU6050 motion sensor, servo motor, and LED indicators. Motion verification and RFID authentication served as the system's two primary verification phases. The user was asked to make a circular motion with the MPU6050 sensor after an authorised RFID card was scanned. The servo motor rotated to unlock if the motion followed the anticipated pattern, and the green LED lit up to show that access had been granted. The red LED illuminated and the servo stayed locked if the motion was improper or the RFID tag was not authorised.

The total system performance demonstrated that the serial connection between the Arduino and Python was an efficient means of communication. When the serial port configuration was correctly matched, RFID scanning and data transfer were dependable, and when delays were appropriately controlled, motion verification was responsive. Unstable servo movement brought on by power constraints and irregular sensor readings as a result of MPU6050 noise were among the practical difficulties. By giving the servo an external power source and introducing a brief lag between serial transmissions, these problems were lessened.

This experiment highlighted the importance of real-time data synchronization and reliable communication between microcontroller and computer-based processing. The use of two-layer verification (RFID and motion gesture) significantly improved the security and functionality of a traditional access control system.

9.0 Conclusion

The experiment's goal of combining Arduino and Python to build and implement a motion-verified RFID access control system was accomplished. To create a completely working prototype, several components such as RFID, MPU6050, servo motor, and LEDs were successfully integrated. The system demonstrated hardware and software cooperation by granting or denying access based on authorised RFID cards and accurate motion detection.

All things considered, the experiment demonstrated how integrating sensors and actuators via serial communication may improve system security and intelligence. Additionally, the experiment yielded important information about serial data processing, hardware interface, and real-time system integration in mechatronics applications.

10.0 Recommendations

There are several improvements that can be done to further boost the system's dependability and performance. First, by combining accelerometer and gyroscope data from the MPU6050 sensor, the gesture detection technique may be enhanced. Using filtering methods like a Kalman or complementing filter would reduce noise and result in motion data that are smoother and more accurate. Second, a steady and separate 5V source for the servo motor and sensors should be included in the power supply configuration to reinforce it. By doing this, servo jittering and voltage dips that can happen when all parts share the same power from the Arduino would be avoided.

It is also advised to have an access logging function to document each RFID scan and motion verification outcome. By enabling users to monitor access history and identify any unauthorised attempts, this will improve system security. Accurate command and data transfer would also be ensured by optimising serial communication between the Arduino and Python, for example, by modifying the baud rate and incorporating simple error-checking techniques. Additionally, creating a basic Python graphical user interface (GUI) that shows RFID information, motion feedback, and real-time access status might enhance user involvement. Finally, wireless connection modules like Bluetooth or Wi-Fi might be added to the system to allow for remote control and monitoring. Along with improving system functioning, these enhancements would also make the system more scalable and useful for real-world applications.

11.0 References

- RFID RC522 Tutorial with Arduino
  https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/
- Security Access using MFRC522 RFID Reader with Arduino
  https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/

12. Acknowledgments

Special thanks to Dr. ZULKIFLI BIN ZAINAL ABIDIN and Dr. WAHJU SEDIONO for their guidance and support during this experiment.

Student's Declaration

Certificate of Originality and Authenticity

We hereby certify that we are collectively responsible for the work presented in this report. The content reflects our original work, except where specific references or acknowledgements have been made. No part of this report has been completed or submitted by individuals or sources not identified within.

Furthermore, we confirm that this report is the result of a collaborative effort, with contributions made by all group members. The extent of each individual's contribution is documented within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Name: AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI	Read	[/]
Matric Number: 2310663	Understand	[/]
Signature: SYAHMI	Agree	[/]

Name: ALYA SYAFINA BINTI ABDUL RAZAK	Read	[/]
Matric Number: 2218084	Understand	[/]
Signature: SYAF	Agree	[/]